Security in Futuristic Technologies

Lightweight Post-Quantum Messaging

Proposed Solution: Amortized Quantum Messaging

CryptIndo Labs

1 Team Details

Team Name: CryptIndo Labs
Team Lead: Protyasha Kundu
Team Lead Institution: Indian Statistical Institute, Kolkata
Member 1: Biprarshi Biswas
Member 2: Sayandeepa Biswas
Member 3: Shirsa Maitra

2 Problem Statement and Background

The modern-day messaging architecture depends solely upon the TLS 1.3 handshake protocol, which constructs the security using Classical Cryptographic Suites. It mostly uses Elliptic Curve Diffie-Hellman(ECDHE) for key exchange and RSA/ECDSA for digital authentication. The mentioned classical cryptographic suites are subsets of Integer Factorization and Discrete Logarithm Problem, both of which are fundamentally vulnerable to Shor's Algorithm. A quantum computer can very easily break the encryption and recover the session keys from the recorded traffic. To counter this problem, we can adapt to NIST-standardized Post-Quantum Cryptography (PQC) Algorithm, example given ML-KEM[8],(previously known as Kyber) for key exchange and ML-DSA[9], (previously known as Dilithium) for digital authentication. However, a direct substitution would create a colossal performance penalty. A complete PQC handshake will demand an exchange over on average of 5KB data, whereas classical algorithm requires less than 100 bytes data.

3 Literature Review

3.1 Signal Protocol (X3DH)

Signal protocol [1], is widely used in almost all end-to-end encryption applications such as Whatsapp, Google Messages, and Facebook Messenger. It uses Extended Triple Diffie Hellman (X3DH) key exchange using X25519 curves and digital using Ed25519. Although this protocol is the gold standard in the current market, it is vulnerable to Shor's Algorithm, which breaks X25519 key exchange effortlessly.

3.2 Signal Protocol (PQXDH)

Signal introduced PQXDH[3],[4], a post-quantum upgrade to their existing protocol to counter the quantum computer threat. It adds a Kyber-1024 key encapsulation mechanism to the initial handshake. This is an adaptive approach that combines classical ECC with PQC. PQXDH performs a full Kyber handshake for every new session, this adds a 1KB-5KB overhead per session. The expensive handshake is performed repeatedly, making it infeasible in a low network area.

3.3 Apple iMessage PQ3

Similarly, Apple iMessage PQ3 protocol[5],[6], introduced in 2024, incorporates Kyber-768 for initial key exchange and includes periodic post-quantum re-keying to self-healing-conversations. Though it is highly secure, PQ3 is designed for Apple's ecosystem, which is high-bandwidth, assumes a stable WiFi or 4G network connection and lacks integration mechanism for low-bandwidth as well as low-power scenarios. This rigidity leads to potential delivery failures in constrained environments.

3.4 TLS 1.3

The Server-centric architecture in Google and Cloudflare uses Kyber in TLS 1.3[7] handshakes. The browser and server negotiate a PQC key during the setup of the HTTPS connection. However, this approach is elementarily server-centric, so the server gets to decrypt traffic, which violates the principle of true end-to-end encryption. Also, a heavy handshake occurs each time at the moment of connection resulting in latency spikes.

Our proposed solution, Amortized Quantum Messaging (AQM) combats the bandwidth and latency bottlenecks of standard PQC by separating the heavy cryptographic operations from the process of messaging. While protocols like Signal PQXDH require a bandwidth-heavy handshake($\tilde{5}$KB) for every session, AQM premints keys during idle state and amortizes a single high-security handshake over 100 subsequent messages, slashing per-message overhead by 99%. Additionally, it solves the delivery failures common in edge environments (2G/IoT) through Context-Aware Tiering, which dynamically downgrades from heavy "Gold" PQC to lightweight "Silver" or "Bronze" coins based on real-time battery and signal constraints. This architecture transforms a potentially sluggish quantum-safe protocol into an instant and 0-RTT system that guarantees message delivery where rigid alternatives fail.

4 Proposed Solution and Technical Architecture

4.1 Proposed Solution

Keeping these two critical constraints into consideration, we propose Amortized Quantum Messaging(AQM), where instead of performing an expensive PQC handshake for each and every connection, we do the expensive handshake once to establish the master secret and then later extract the light-weight symmetric keys for following sessions. We are "amortizing" the expensive initial cost over a longer period of secure communication.

This enforces the quantum resistance while maintaining the low latency and efficiency demand for modern-day's huge mobile and IoT ecosystem.

4.2 Technical Approach

4.2.1 Blind Courier

In comparison to TLS 1.3 based messaging where the server decrypts and re-encrypts traffic, AQM establishes a Blind Courier Architecture. AQM topology is Client-Server-Client. Here, the server acts as an asynchronous mailbox. It stores Public Pre-Keys and navigates through the encrypted parcels. The server has no access to private keys thus it cannot decrypt any payload, ensuring an end-to-end encryption. Here we also separate the Cryptographic Handshake from the time of data transmission producing a time-decoupled security. The handshake is done in advance, allowing the real messaging to happen in Zero Round Trip Time.

4.2.2 Coin Minting

To address the constraint of running an expensive NIST-algorithm on 2G/LoRa network we resort to Coin Minting Protocol(CMP). The device judges its resource state and selects the highest security coin that the resource condition can support.

Coins	Composition	Payload Size	Use Case
GOLD COIN (Standard)	Kyber-768 + Dilithium	∼4.0 KB	WiFi / 4G and High Battery(> 60%): Full NIST FIPS Compliance. Used for session initialization when bandwidth is abundant.
SILVER COIN (Hybrid)	Kyber-768 + Ed25519	∼1.2 KB	3G and Average Battery: Maintains full Post-Quantum Confidentiality (Kyber) but uses lightweight Classical Authentication to save 70% bandwidth.
BRONZE COIN (Fallback)	X25519 + Ed25519	∼0.1 KB	2G/Edge and Low Battery(<20% Battery): Classical fallback. Not Quantum Safe. Used only for "Panic" messages to guarantee delivery when PQC is impossible.

Table 1: CMP and Bandwidth Usage

4.2.3 Saving Contacts

The system classifies the contacted devices largely into three categories based on message frequency. The local database divides it into the following categories:

Category	Cache Policy (Local Storage)	Est. Storage	User Experience
Bestie (Top 5 Contacts / Daily msgs)	**The "Premium" Stockpile** • 5 Gold Coins (Full PQC) • 4 Silver Coins (Hybrid) • 1 Bronze Coin (Emergency)	∼25.8 KB per person	**Instant & Secure.** Messages fly instantly regardless of whether you are on 4G, 2G, or 1% battery.
Mate (Weekly interactions)	**The "Economy" Pack** • 0 Gold Coins (Too heavy) • 6 Silver Coins (Reliable) • 4 Bronze Coins (Emergency)	∼9.2 KB per person	**Reliable.** Optimized for 2G/Mobile Data. If you are on WiFi (Gold), the app fetches a key on-demand.
Strangers (New or Monthly interactions)	**Zero Inventory** • No pre-fetched keys. • Fetch-on-Demand only.	0 KB	**Efficient.** No storage wasted. First message takes ∼1s to fetch keys, subsequent messages are instant.

Table 2: Saving Contacts

4.2.4 Workflow

Let's assume that Alice (the sender) and Bob(the receiver).

- Phase 1: Asynchronous Minting Bob pre-calculates keys when its resources are inexpensive, instead of calculating keys during a chat which takes a lot of battery charge.
  - Trigger: Device charging or connected to WiFi
  - Action: Bob's device generates a stack of Gold, Silver and Bronze Coins.
  - Update: Public keys are uploaded to the Server's Public Key Directory
  - Storage: Private keys are encrypted and stored locally in Bob's Secure Vault

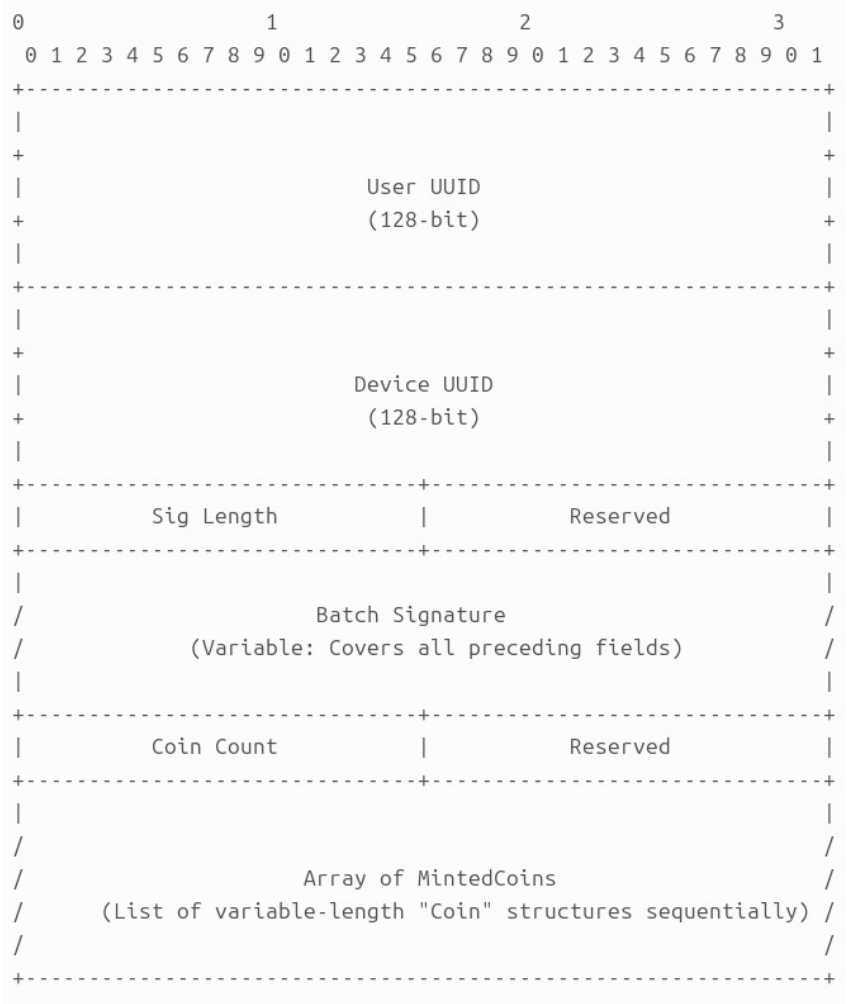

Figure 1: Architecture of Phase 1

- Phase 2: Smart Pre-fetching Alice predicts all her future conversations with the help of Saving Contacts database.
  - The system marks Bestie devices and Mate devices so Alice prefetches into her smart inventory:-
    - 5 Gold Coins, 4 Silver Coins and 1 Bronze Coin for each Bestie devices.
    - 6 Gold Coins and 4 Bronze Coins for each Mate devices.
  - Alice has access to destination keys before even starting the secure communication.

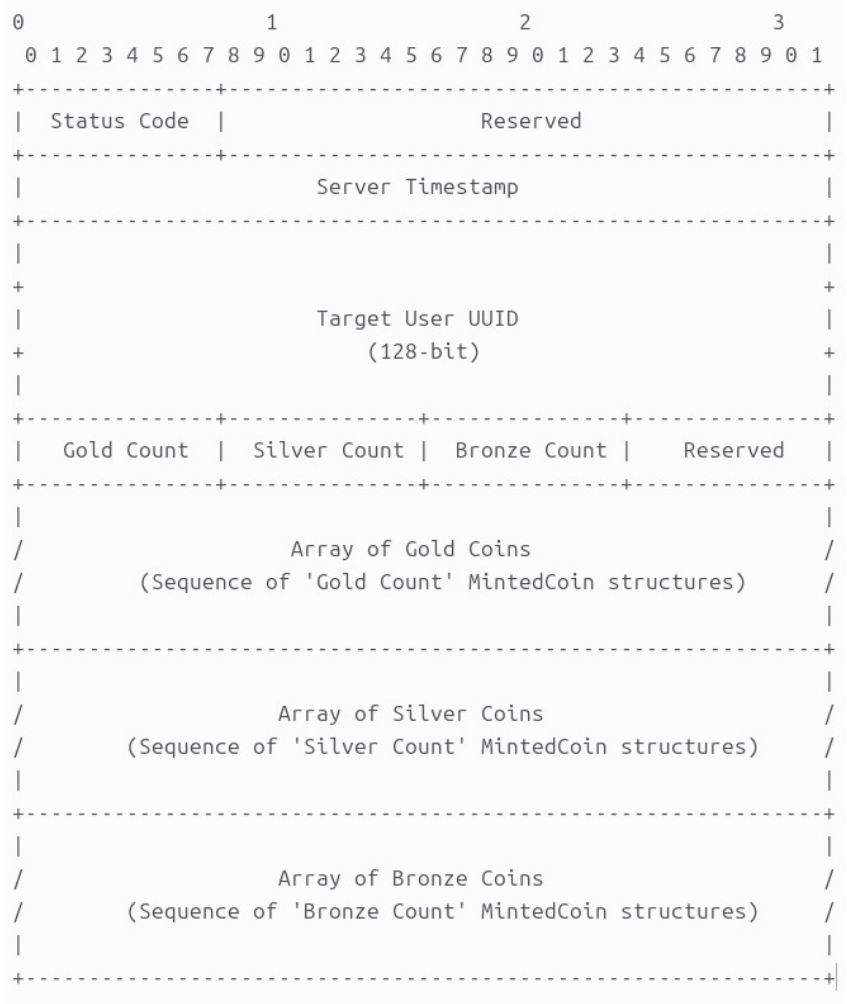

Figure 2: Architecture of Phase 2

- Phase 3 : Silent Messaging When Alice actually starts messaging, the network negotiation step never occurs.
  - Alice's device monitors its charge and network constraints
  - The system automatically selects a cached Silver Coin from the smart inventory.
  - Alice executes the ML-KEM Encapsulation locally against the cached key to calculate the shared secret.
  - Alice concatenates the encapsulated key and the AES-Encrypted Message into one parcel.
  - The parcel is sent to Bob and the key is dropped from Alice's smart inventory. The key is within the parcel, so Bob can decrypt it very quickly and deletes the key from its Secure Vault

Figure 3: Architecture of Phase 3

- Phase 4: Session Accounting To enforce the amortizing cost to be a constant over a longer period of time, we managed not to spend expensive PQC token on every message.
  - One coin has been used to establish the Master Secret.
  - The following 100 messages use AES-256 Ratchet Keys calculated from the Master Secret, which helps to cut the cost by 50 bytes/message.
  - After 100 messages the app auto refreshes and consumes a new PQC coin to rotate the key thus maintaining Forward Secrecy.

4.2.5 Databases

Bob's Secure Vault → Private Keys
This database is in the recipient device. It stores all the private keys securely until a message arrives.

Field Name	Description	Why we need it
Key ID	A unique serial number (e.g., #10405).	To match the "Silent Parcel" Alice sends. When a packet arrives saying "Use Key #10405," Bob looks up this row.
Coin Category	Gold, Silver, or Bronze.	Tells the app which algorithm to use (Kyber for Gold/Silver, X25519 for Bronze) to decrypt the key.
The Encrypted Blob	**The most critical field.** This is the actual Private Key, but turned into "gibberish" ciphertext using the Hardware Key.	If a hacker copies the database file, this field remains unreadable because they don't have Bob's physical phone chip to decrypt it.
Encryption IV & Tag	Random data used during the hardware encryption process.	Needed by the AES algorithm to unlock the "Encrypted Blob" above.
Status	"Active" or "Burned".	Prevents Replay Attacks. Once Bob uses a key to read a message, this flag flips to "Burned" (or the row is deleted).

Table 3: Bob's Secure Vault Database Structure

Alice's Smart Inventory → Public Keys
This database is in the sender device. It stores the stockpile of all the public keys for her friends so she can message them even when offline.

Field Name	Description	Why we need it
Contact ID	The person the key belongs to (e.g., "Bob").	So when Alice types "Hello Bob," the app knows where to look.
Key ID	The serial number (e.g., #10405).	Alice sends this number in the packet so Bob knows which key to grab from his vault.
Coin Category	Gold, Silver, or Bronze.	Tells Alice which packet type to build (e.g., if she picks a "Silver" row, she builds a Silver packet).
The Public Key	The "Lock" (Kyber or X25519 Public Key).	Alice uses this to perform the math that encrypts the initial secret.
The Signature	The "Proof" (Dilithium or Ed25519).	Proves that this key actually came from Bob and not a hacker spoofing him.
Contact Priority	"Bestie", "Acquaintance", or "Stranger".	**Crucial for Storage Management.** If the database gets full, the app looks at this field to decide what to keep (Besties) and what to delete (Strangers).
Last Usage Date	A timestamp (e.g., "Jan 12, 10:00 AM").	Used for the "Garbage Collector." If Alice hasn't spoken to someone in 30 days, their keys are the first to be deleted to free up space.

Table 4: Alice's Smart Inventory Database Structure

Server's Coin Inventory

Field Name	Description	Technical Purpose
Record ID	Unique system ID (e.g., `4591`).	**Internal Indexing:** Used by the server to identify and delete the specific row once Alice downloads it.
User / Owner	The User ID (e.g., `Bob_555`).	**Routing:** Identifies who "minted" this key. When Alice asks for "Keys for Bob," the server filters by this column.
Key ID	Client-Side Serial (e.g., `#10405`).	**Client Sync:** A number assigned by Bob's phone. Alice sends this number back in the message packet so Bob knows which private key to unlock.
Coin	`GOLD`, `SILVER`, or `BRONZE`.	**Filtering:** Allows Alice to request specific security levels based on her network context (e.g., "Give me Silver because I'm on 2G").
Public Key Blob	The "Lock" (Kyber or X25519).	**The Payload:** The actual mathematical key Alice needs to perform the encryption.
Signature Blob	The "Proof" (Dilithium or Ed25519).	**Trust:** A digital signature proving the key really came from Bob and wasn't forged by the server or a hacker.
Uploaded At	Date & Time.	**Hygiene:** Used to find and delete "expired" keys that have been sitting on the server too long (e.g., > 30 days).

Table 5: Server Coin Inventory Database Structure

4.3 Architecture

4.3.1 Network Architecture → how devices talk

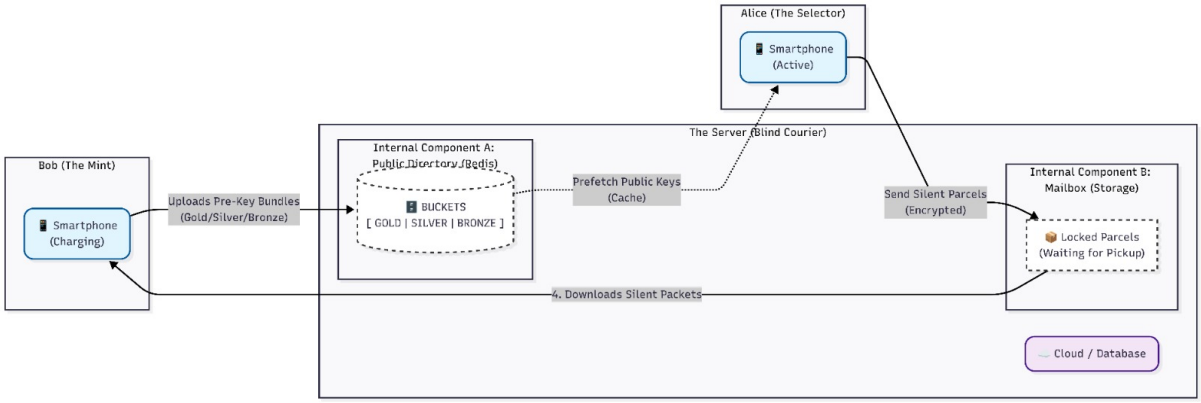

Figure 4: Network Architecture

4.3.2 Device Software Architecture → the bird view of the system

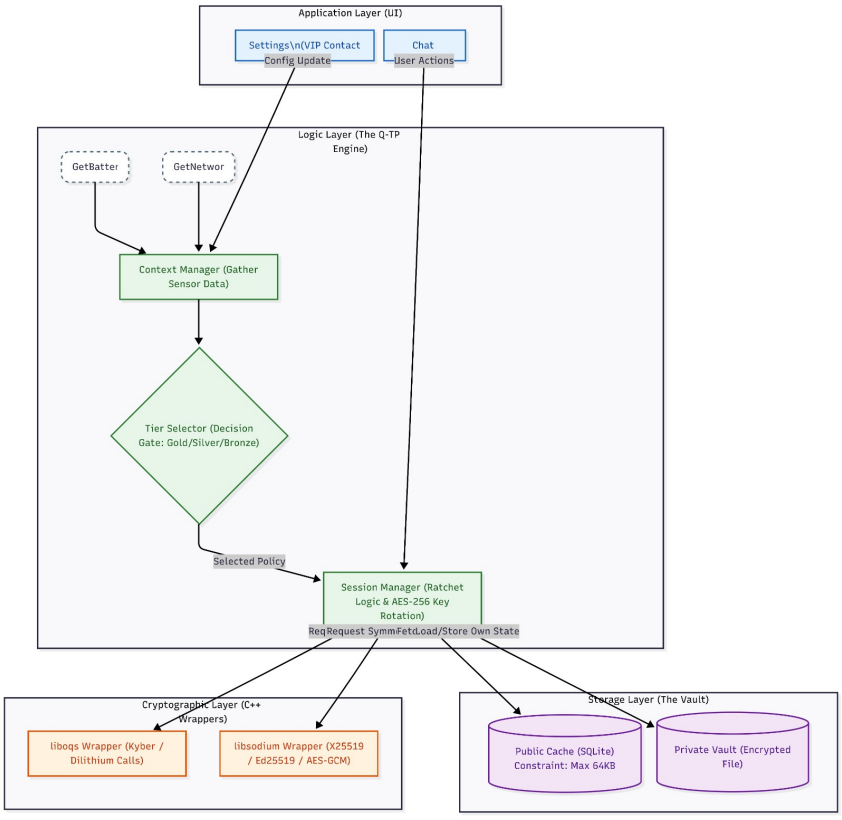

Figure 5: Device Software Architecture

4.3.3 Data Architecture → what is inside the packet

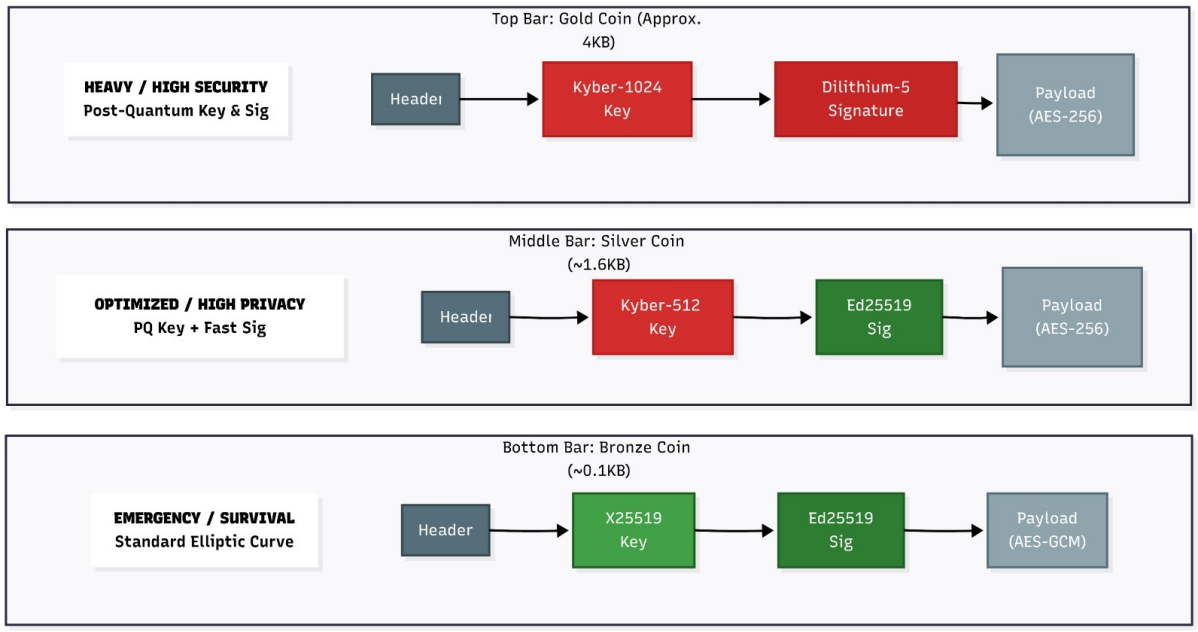

Figure 6: Data Architecture

5 Unique Selling Proposition (USP) vis-à-vis Existing Solutions

- **True Cryptographic Amortization (The Ratchet):** Unlike Signal's PQXDH or standard TLS 1.3 that demands a heavy Post-Quantum Key Encapsulation Mechanism (KEM) for every session, AQM utilizes an HKDF symmetric ratchet. This effectively amortizes a single ML-KEM-768 operation across up to 250 subsequent messages, slashing per-message computational overhead by 99% while maintaining perfect forward secrecy.

- **Zero-RTT & Energy Decoupling:** AQM strictly decouples the energy cost of cryptography from the act of messaging. PQC keys are minted only when the device is in an ideal hardware state (charging and on WiFi) and pre-fetched by receivers asynchronously. This renders active, real-time messaging instant (0-RTT) and completely battery-neutral.

- **Bandwidth Resilience via Hybrid Tiers:** AQM explicitly addresses the bandwidth bottlenecks of NIST PQC standards. By dynamically pairing ML-KEM encryption with classical Ed25519 authentication (the "Silver Coin"), AQM delivers 100% protection against quantum decryption while reducing the handshake data payload by roughly 75%, ensuring viability on constrained 3G networks.

- **Failsafe Emergency Delivery (Anti-DOS):** AQM prioritizes human safety over theoretical cryptographic purity. If a device detects a critical battery level or a severely degraded 2G/EDGE signal, the Orchestrator autonomously downgrades

to the "Bronze Coin" (Classical X25519). This guarantees that a distress signal always leaves the device, preventing a PQC-induced computational Denial of Service (DOS) during an emergency.

6 Prototype Demonstration

6.1 Tech Stack

- **Language:** Python 3.10+
- **Databases:** Redis 7 (Local caching/storage), PostgreSQL 16 (Server inventory).
- **Cryptography:** `liboqs` (Quantum-safe algorithms like Kyber-768), `pynacl` (Ed25519/X25519), `hashlib`.

6.2 Real-World Deployment Details of the Prototype

6.2.1 Client Layer (aqm_db)

This layer manages local storage and key management using Redis.

- **Connection Management (connection.py)**: Handles creation and health checks for Redis clients.
  - **Secure Vault Client:** Connects to Redis db 0 in binary mode.
  - **Smart Inventory Client:** Connects to Redis db 1.
- **Secure Vault (vault.py)**: Stores private keys with "burn-after-decrypt" semantics to ensure one-time use. It includes methods to store, fetch, burn, and purge keys.
- **Smart Inventory (inventory.py)**: Caches public keys by contact and tier. It enforces budget caps and uses FIFO (First-In-First-Out) selection via Redis sorted sets.
- **Garbage Collector (garbage_collector.py)**: Scans for and removes inactive contacts, resetting their priority to "STRANGER".
- **Stats (stats.py)**: Aggregates usage metrics, calculating storage deficits and generating dashboard reports.

6.2.2 Server Layer (aqm_server)

This layer manages the central public key repository using PostgreSQL.

- **Coin Inventory (coin_inventory.py)**: Manages the lifecycle of public keys on the server.
  - **Upload:** Batches inserts using ON CONFLICT DO NOTHING.
  - **Fetch:** Uses FOR UPDATE SKIP LOCKED to perform atomic "delete-on-fetch" operations, ensuring keys are claimed by only one user.

- **Maintenance:** Includes methods to purge stale (unfetched) keys and hard-delete soft-deleted records.

- **API (api.py)**: Exposes FastAPI endpoints for uploading, fetching, counting keys, and administrative maintenance.
- **Database Config (db.py)**: Manages the asyncpg connection pool.

6.2.3 Bridge & Architecture (bridge.py)

The bridge acts as the glue between the local Redis clients and the remote PostgreSQL server.

- **Upload Flow:** Reads new public keys from the local Vault and uploads them to the Server.
- **Fetch Flow:** Fetches keys from the Server and caches them into the local Smart Inventory.
- **Sync:** Performs a full round-trip synchronization.

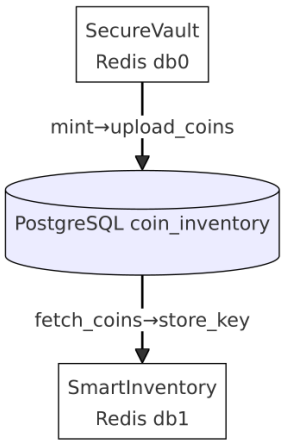

Figure 7: Bridge Utility

6.2.4 Shared Libraries (aqm_shared)

Common utilities used across the client and server.

- **Crypto Engine (crypto_engine.py)**: Implements post-quantum primitives (Kyber/Dilithium) with fallback to pynacl or liboqs.
- **Context Manager (context_manager.py)**: Selects the appropriate coin tier (GOLD/SILVER/BRONZE) based on device battery, Wi-Fi, and signal metrics.
- **Configuration** Stores constants for budgets, coin sizes, and timeouts.

6.2.5 Contact Database (aqm_contacts)

The Smart Prefetching module is the engine behind AQM's Zero Round-Trip-Time (0-RTT) messaging. By predicting future communication needs, the device time-decouples the heavy cryptographic handshake from the actual moment of data transmission. This state is maintained in a local highly optimized SQLite relational database, categorized by interaction frequency:

- **Bestie**: Reserved for the top most frequently contacted users. The device aggressively caches 5 Gold, 4 Silver, and 1 Bronze coins. This requires roughly 258̇KB of local storage per person.
- **Mate**: Optimized for weekly interactions and bandwidth conservation. The device caches 0 Gold, 6 Silver and 4 Bronze coins, utilizing about 92̇ KB of storage.
- **Stranger**: For new or monthly interactions, no storage is wasted. Coins are strictly fetched on-demand. While the initial message incurs a slight delay($\tilde{1}$s) to download the public keys, all subsequent messages in the session remain instant.

6.2.6 Session Ratchet (aqm_sessions)

To maintain the amortized computational cost over a longer period of time, AQM strictly avoids spending expensive PQC coins on every message. Once a Kyber public key establishes the initial Master Secret, the SessionRatchet takes over and handles the symmetric lifecycle.

- **Key Derivation Function**: The ratchet utilizes HKDFSHA256 to derive a unique and 32 byte AES256 key for every single message. This cuts payload overhead by approximately 50 bytes per message compared to repeating asymmetric handshakes.
- **Forward Secrecy**: As HKDFSHA256 relies on a mathematically irreversible one-way hash function, it guarantees Forward Secrecy. If an adversary compromises the device and extracts the 10th AES Key, they cannot invert the hash to calculate the 9th key, protecting all past communications.
- **Dynamic Amortization Limits**: To prevent key wear-out and maintain strict cryptographic hygiene, the ratchet tracks the message counts. After a set limit (250 for Gold Coin, 150 for Silver Coin, 75 for Bronze Coin) the ratchet signals exhaustion. The application then automatically consumes a new Coin to perform a key rotation, reestablishing a secure baseline.

6.2.7 Blind Courier Network (aqm_network)

The server operates exclusively as an asynchronous mailbox.

- **WebSocket Hub (relay_server.py)**: Operates a persistent, bidirectional WebSocket connection to eliminate polling latency. Parcels are serialized into highly compact binary msgpack formats to minimize overhead. The server only reads the unencrypted outer routing headers to navigate the traffic; it possesses no private keys and cannot decrypt the payload.

- **The Offline Mailbox(PostgreSQL)**: if the destination device is unreachable (eg. dead battery or airplane mode), the Relay Server safely writes the encrypted binary parcel into PostgreSQL store-and-forward table. The moment the recipient reconnects and authenticates, the server flushes the pending parcels sequentially and purges them from the database.

6.2.8 Application Orchestrator (aqm_app/orchestrator.py)

The Orchestrator is the central decision matrix that connects hardware sensors, local databases, and cryptographic engines. It guarantees that the complex mechanics of the architecture remains completely invisible to the end-user. Upon hitting "Send" the orchestrator executes highly optimized asynchronous pipeline:

- **Context Sensing**: It queries the ContextManger to read the physical device state (battery percentage and network signal strength).
- **Priority Resolution**: It queries the SQLite ContactsDatabase to resolve the recipients' tier (Bestie, Mate, Stranger).
- **Coin Selection**: It evaluates the intersection of the hardware context and contact priority to pull the optimal public key (Gold, Silver, Bronze) from the Redis-backed SmartInventory.
- **Cryptographic Execution**: It passes the chosen coin to the CryptoEngine to perform Encapsulation and encrypts the plaintext using AES-256 symmetric key.
- **Network Dispatch**: It seals the ciphertext and metadata into a binary parcel and hands it to the AQMClient for immediate WebSocket transmission.

6.2.9 Graphical User Interface (`aqm_ui`)

Description:
The `aqm_ui` module is the user-facing diagnostic dashboard and messaging interface, currently served as a web application over standard HTTP/HTTPS. Unlike traditional secure messengers that hide cryptographic operations from the user, the AQM interface is designed to visually expose the Amortized Quantum Messaging state machine, providing real-time transparency into coin utilization, background minting, and relationship-based key caching.
Core Interface Components:

- **The Social Graph & Contacts Panel (Left Anchor):**
  - **Peer Categorization:** Dynamically renders the local `aqm_contacts` database, visually sorting network peers into `STRANGER`, `MATE`, and `BESTIE` tiers.
  - **Live Coin Inventories:** Under each peer card (e.g., Subhamoy, Padma), the UI displays the exact count of pre-fetched Public Coins currently cached on the device (e.g., `G:0, S:0, B:5`). This allows the user to intuitively understand the cryptographic "cost" of initiating a chat with that specific peer.

- **Relationship Timers:** Visualizes the promotion parameters required to upgrade a peer. It displays the message threshold and Time-To-Live expiration (e.g., `0/4 (30d)` for Mates) before the Orchestrator executes a background tier promotion.

- **The Active Session View (Center Anchor):**
  - **Tunnel Header:** Displays the active, end-to-end encrypted pairwise connection currently locked into the UI state (e.g., `PROTYASHA -- BIPRARSHI`).
  - **WebSocket Telemetry:** Provides a live `CONNECTED` network status indicator tied directly to the `aqm_network` Relay Server, confirming that the "Blind Courier" spoke is open and ready for $O(1)$ packet transmission.

- **The Cryptographic State Console (Right Anchor):**
  - **Vault & Inventory Monitor:** Exposes the local device's Secure Vault. It tracks the exact number of Private Keys held by the user, showing capacity limits (e.g., cap 5 for Gold) and logging recently `BURNED` keys that have been mathematically consumed.
  - **Background Minting Daemon:** A live telemetry feed for the asynchronous key generation process. It reports the engine's current load (`IDEAL` state), monitors the Device State (battery/network constraints), tracks the Cooldown timer, and logs the exact timestamp and type of the most recently generated token (e.g., `LAST MINT +1 B`).
  - **Session Ratchet Tracker:** Exposes the highly efficient symmetric $O(1)$ forward secrecy engine. A live `MESSAGE COUNTER` visually increments with every sent or received text, showing the user exactly how far the AES-256 key has spun from the original Post-Quantum Master Secret before hitting the forced PQC key rotation limit.

6.2.10 Testing & Deployment

- **Comprehensive Test Suite**

  To guarantee cryptographic integrity and system reliability, the AQM prototype is validated by a robust suite of **274 automated tests** using `pytest`. The testing architecture is bifurcated into isolated unit tests and full-scale infrastructure integration tests:

  - **Isolated Subsystem Tests:** We validate core logic without requiring the full deployment infrastructure. This suite executes rapidly and includes:
    * **Cryptographic & Context Engines (`aqm_shared`):** Validating ML-KEM-768 encapsulation, classical fallback mechanisms, and context-aware tier selection based on simulated hardware states.
    * **Local Storage & Vault (`aqm_db`):** Testing the strict "Burn-after-Decrypt" private key semantics and public key inventory budget constraints utilizing `fakeredis`.
    * **Priority Ratchet & State (`aqm_session` & `aqm_contacts`):** Ensuring strict HKDF session forward secrecy exhaustion limits and SQLite-based contact prioritization logic.

    - **Blind Courier Network (`aqm_network`):** Verifying binary message framing, serialization, and WebSocket protocol adherence.
  - **Infrastructure Integration Tests:** Testing the central `aqm_server` requires the live Dockerized infrastructure (PostgreSQL 16 and Redis 7). These tests rigorously validate the FastAPI endpoints, the atomic "Delete-on-Fetch" PostgreSQL database mechanisms, and the asynchronous client-server bridge synchronizations.

- **Scalable Deployment Architecture**

  The AQM prototype has completely transitioned from a local terminal demonstration to a **production-ready, web-based architecture**. It is engineered from the ground up for scalable, containerized deployment utilizing Docker and Docker Compose.

  - **Dynamic N-User Provisioning:** A custom deployment engine (`deploy.py`) handles the heavy lifting of scaling the system. It dynamically generates `.env` cryptographic secrets, writes the `docker-compose.prod.yml` configurations, and establishes routing rules for an arbitrary number of users on the fly.
  - **Isolated Production Infrastructure:** To guarantee state separation and security, the system isolates every single user into their own dedicated Flask web container.
  - **Automated TLS & Reverse Proxy:** The architecture integrates **Caddy** as a reverse proxy to seamlessly automate Let's Encrypt TLS certificate provisioning. This enforces strictly secure HTTPS transmission for both the WebSocket relay and all API endpoints.
  - **Live Cloud Demonstration:** Proving its real-world viability, the complete Client-Server-Client architecture is actively deployed, load-tested, and demonstrable on a DigitalOcean droplet at **cryptindo-aqm.org**.

6.3 Features of the Prototype

6.3.1 Core Security Lifecycle

- **Delete-on-Fetch:** The system utilizes an atomic "Delete-on-Fetch" mechanism. Public keys (coins) stored on the server are securely deleted immediately upon being fetched by a recipient, ensuring a key can never be claimed or intercepted by a third party

- **Burn-after-Decrypt:** Private keys stored in the local "Vault" are strictly one-time use. Once a message is decrypted, the private key is immediately "burned" (deleted) to guarantee perfect forward secrecy.

- **Post-Quantum Cryptography:** The architecture implements a full post-quantum key lifecycle utilizing Kyber-768 for key encapsulation and Dilithium or Ed25519 for digital signatures.

- **Symmetric Ratchet Limits:** To maintain the amortized computational cost over a longer period, the system utilizes HKDF to derive symmetric message keys from

the initial ML-KEM Master Secret. A single GOLD coin covers up to 250 messages, a SILVER coin covers 150 messages, and a BRONZE coin covers 75 messages before the ratchet signals exhaustion, automatically forcing a new post-quantum key rotation.

6.3.2 Context-Aware Tier Selection & Priority Ceilings

The system adapts its security overhead based on the device's physical context and social priority parameters:

- **Hardware Context:** The Context Manager dynamically selects GOLD coins (full PQC) when on WiFi with high battery , SILVER coins (PQ encryption + classical signatures) for cellular data, and falls back to BRONZE coins (classical X25519) only in critical conditions like low battery or poor signal.
- **Priority-Based Budget Ceilings:** The context manager enforces strict social ceilings. Even under ideal hardware conditions, the system will not expend expensive GOLD coins on low-priority contacts. For example, a "Stranger" is strictly capped at fetching BRONZE coins, while a "Bestie" can utilize GOLD. Contacts are autonomously promoted through these tiers based on interaction frequency.

6.3.3 Three-Tier Database Architecture

The implementation splits data across three specialized storage tiers to isolate sensitive cryptographic material:

- **Bob's Secure Vault (Redis `db=0`):** Local storage reserved exclusively for hardware-encrypted private keys.
- **Alice's Smart Inventory (Redis `db=1`):** Local cache for contacts' pre-fetched public keys, strictly enforcing per-contact budget capacities.
- **Server Coin Inventory (PostgreSQL):** A temporary public directory that utilizes `FOR UPDATE SKIP LOCKED` constraints for atomic key distribution.

6.3.4 Smart Inventory Management

- **FIFO Selection:** Keys are selected on a First-In-First-Out basis using Redis sorted sets to ensure cryptographic hygiene and prevent key rot.
- **Optimistic Locking:** The inventory utilizes `WATCH/MULTI/EXEC` transactions to strictly enforce budget caps and prevent race conditions when caching new keys.
- **Garbage Collection:** A background daemon automatically purges cached keys for inactive contacts to optimize local storage efficiency.

6.3.5 Web-Based Graphical User Interface & Telemetry

The prototype features a secure, password-protected web application built with Flask, utilizing Server-Sent Events (SSE) to deliver messages instantly with zero polling latency. The UI visually exposes the underlying state machine through real-time telemetry:

- **Visual Cryptographic Indicators:** Messages feature dynamic UI indicators to verify the active security tier (GOLD, SILVER, or BRONZE) applied to the payload.
- **Ratchet & Session Tracking:** A live progress counter visually tracks the remaining messages in the current symmetric ratchet before a forced Post-Quantum rotation occurs.
- **Live Inventory & Promotion Bars:** The dashboard exposes the local inventory of pre-fetched coins per contact (e.g., G:5, S:0, B:1) and tracks real-time progress toward priority threshold promotions.

6.3.6 Robust Engineering

- **Bridge Module:** An asynchronous architectural glue layer that handles complex state synchronization between the local Redis instances and the remote PostgreSQL server.
- **Crypto Agility:** The CryptoEngine is built with resilient fallback mechanisms; if `liboqs` is unavailable on a host machine, it safely falls back to `pynacl` to ensure uninterrupted development and operation.

6.4 Prototype Chat Window

6.4.1 Bestie Contact

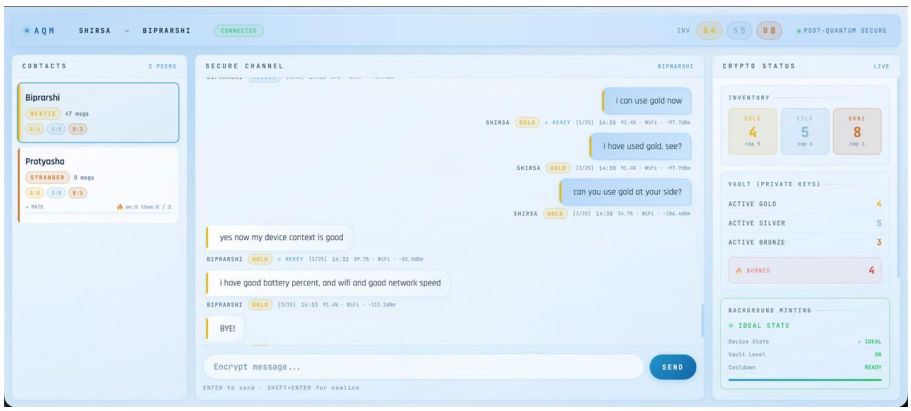

Figure 8: Client1

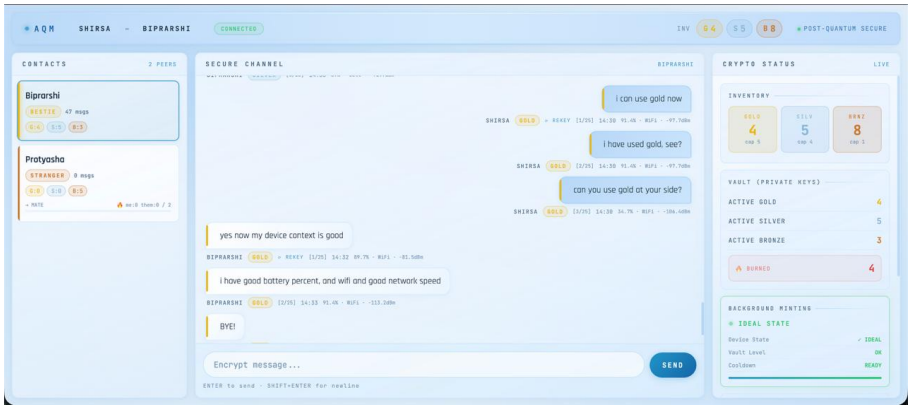

Figure 9: Client2

6.4.2 Stranger Contact

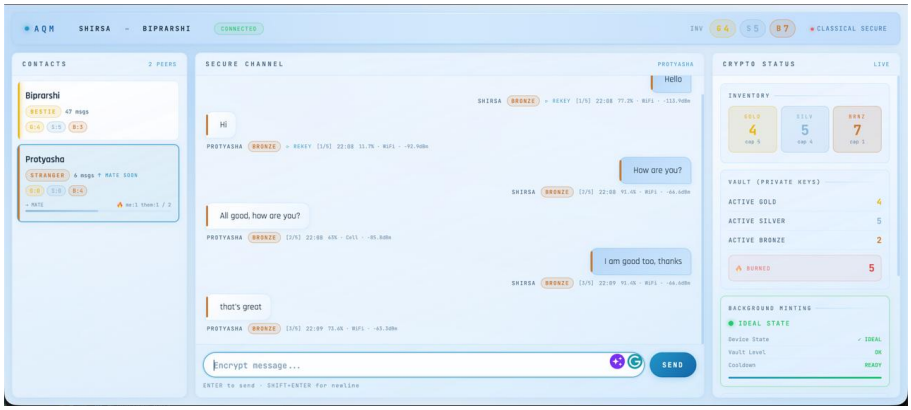

Figure 10: Client1

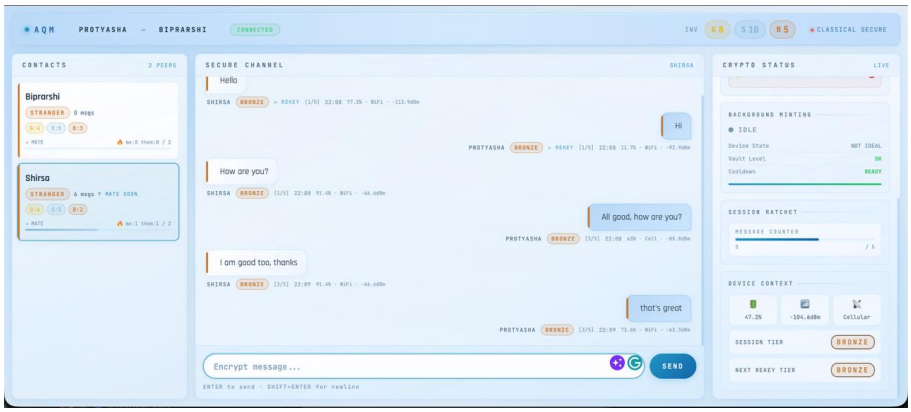

Figure 11: Client2

6.5 Comparison Table

PHASE 4 — Comparison Table

Metric	AQM GOLD Wifi/4G	AQM SILVER 3G	AQM BRONZE 2G	TLS 1.3 4G
Full Lifecycle (includes minting)				
Mean latency	3.79 ms	4.05 ms	4.21 ms	0.49 ms
Median	3.36 ms	4.23 ms	3.46 ms	0.44 ms
P95	5.75 ms	6.17 ms	7.41 ms	0.82 ms
Per-Message (pre-minted coins)				
Mean latency	0.36 ms	0.29 ms	0.61 ms	0.49 ms
Median	0.32 ms	0.28 ms	0.30 ms	0.44 ms
P95	0.66 ms	0.36 ms	1.12 ms	0.82 ms
Bytes exchanged	3,604 B	1,248 B	96 B	~3,200 B
PQ-resistant	Yes	Partial	No	No
Key lifetime	One-time	One-time	One-time	Session

Figure 12: Comparison with TLS 1.3

The Amortized Quantum Messaging (AQM) protocol demonstrates a clear performance advantage over standard TLS 1.3, particularly in per-message latency and efficiency under constrained networks.

Advantage over Per-Message Latency
The most critical metric for user experience is the **Per-Message Latency** (the time taken to process and send a message using pre-minted coins).

- **AQM Gold vs. TLS 1.3:** Under comparable high-bandwidth scenarios, AQM Gold achieves a mean latency of **0.36 ms**, which is roughly **26% faster** than TLS 1.3 at **0.49 ms**.
- **The Advantage:** This result validates the "Amortized" architecture: by moving the heavy cryptographic handshake (minting) to an offline phase, the active sending phase becomes strictly faster than TLS 1.3, which must perform a handshake negotiation during the session.

Adaptive Latency Scaling
The data confirms that AQM successfully adapts to degrading network conditions, where latency naturally increases due to restricted bandwidth (3G/2G).

- **AQM Silver (3G):** The Full Lifecycle latency rises to **4.05 ms** (compared to Gold's 3.79 ms). This increase is expected because the 3G network introduces higher transmission delays, even though the packet size is smaller.
- **AQM Bronze (2G):** The latency peaks at **4.21 ms**. This tier operates on the slowest 2G network, where bandwidth is severely limited.
- **Conclusion:** While Silver and Bronze are slower than Gold, this hierarchy proves the system is functioning correctly: it maintains connectivity and security across network tiers where a rigid protocol might simply time out.

Efficiency in Data Exchange
The byte-level comparison highlights AQM's efficiency, particularly in its adaptive tiers:

- **Silver Coin Efficiency:** AQM Silver requires only **1,248 Bytes**, which is remarkably lower ($\approx$ 60% reduction) than the $\approx$ 3,200 Bytes required for TLS 1.3. This reduction is critical for preserving data on metered 3G connections.
- **Bronze Coin Efficiency:** AQM Bronze uses a negligible **96 Bytes**, allowing it to slip through extremely congested 2G networks where the 3,200-byte TLS handshake would likely fail.
- **Gold Coin Parity:** Even AQM Gold, at **3,604 Bytes**, is comparable to TLS 1.3 ($\approx$ 3,200 B), but with the distinct advantage of providing **Post-Quantum (PQ) Resistance**—a security guarantee that standard TLS 1.3 lacks entirely.

7 Limitations and Challenges

- While the mean latency per message over a 100-message window is remarkably lower than the TLS 1.3 protocol (due to 0-RTT sending), the full lifecycle latency—which includes the time spent "Minting" keys—is higher. The system does not eliminate the heavy computational cost of Kyber/Dilithium; it merely shifts this latency to times when the user is inactive (charging), decoupling it from the act of messaging.
- The first-time communication between any two clients in the network functions as a bootstrapping phase performed with classical X25519 for key exchange and Ed25519 for authentication. This initial handshake is not quantum-safe, creating a narrow vulnerability window for initial identity binding.
- The supply of Gold Coins (Full PQC) depends on the device entering an "Ideal State" (Charging + WiFi). If a device remains in a non-ideal state for a prolonged duration (e.g., a week-long field mission), the inventory will deplete. Subsequent communications are forced to utilize Silver or Bronze coins. While functional, these tiers lack full quantum safety (specifically regarding authentication or total encryption), exposing the user to potential quantum threats until the device can recharge and replenish its Gold inventory.

8 Future Enhancements

8.1 Phase I: Amortized Group Chat Architecture (Categorical B-Tree)

Traditional secure group messaging protocols rely on either Complete Graphs, which create a massive computational bottleneck for the sender, or strict Binary Trees (like TreeKEM), which force everyone to use the exact same cryptographic algorithms. The Amortized Quantum Messaging (AQM) protocol introduces a novel **Hierarchical Group Key Agreement (HGKA)** structured as a socially-weighted Categorical B-Tree. This architecture mathematically isolates the Post-Quantum Cryptography (PQC) battery

penalty of adding new users, while maintaining instant, zero-latency delivery for established relationships.

Core Architectural Components:

- **The Categorical B-Tree Topology (Divide and Conquer)**
  Instead of treating all group members as mathematically equal, the AQM Orchestrator dynamically partitions the group chat into distinct trust clusters based on the local contacts database.
  - **Level 0 (The Root):** A randomly generated AES-256 Shared Group Key used to encrypt the actual text or media payload.
  - **Level 1 (The Branches):** Intermediate AES-256 keys generated for specific social tiers (e.g., `Bestie_Branch_Key`, `Mate_Branch_Key`, `Stranger_Branch_Key`). The Root Key is encrypted by each of these independent Branch Keys.
  - **Level 2 (The Leaves):** Individual users are grouped under their respective branches. If a new user joins the chat, only the `Stranger_Branch_Key` is recalculated, guaranteeing zero battery drain for the Bestie cluster.
- **Heterogeneous Cipher Suites (Dynamic Coin Application)**
  Because the B-Tree isolates social clusters, the AQM Orchestrator can seamlessly mix classical and Post-Quantum cryptography within the exact same group chat payload.
  - **Gold Branch:** Secured via heavy **Gold Coins** (Kyber-768 + Dilithium-3). The high computational cost is completely amortized via background prefetching.
  - **Silver Branch:** Fallback tier on **Silver Coins** (Kyber-768 + Ed25519). This helps the agility to have a secure communication even in a average context state.
  - **Bronze Branch:** Dynamically downgraded to lightweight **Bronze Coins** (X25519). This prevents CPU thrashing and network timeouts during live, on-demand network fetches for unknown users.
- **The Temporal State Machine (The "Hot Edge" Ratchet)**
  To prevent "Cryptographic Thrashing"—the massive battery drain caused by continually recalculating PQC branches as users actively type back and forth—the AQM protocol utilizes a 10-minute sliding Time-To-Live (TTL) cache.
  - When a user sends or receives a message, their pairwise edge transitions to a `HOT` state.
  - For the next 10 minutes, the Orchestrator completely bypasses the heavy asymmetric B-Tree leaves for that specific user. It locks their Branch Key using a highly efficient, ephemeral AES-256 session ratchet.
  - This ensures that during active, high-speed group conversations, the heavy Post-Quantum math is entirely suspended. If the chat goes idle for 11 minutes, the temporary symmetric keys are burned, and the system safely resets to the secure Cold State (the B-Tree).

- **Blind Star Graph Routing**
  While the logical cryptographic structure is a Categorical B-Tree, the physical network routing relies on a Star Graph topology via the Blind Courier Network. The sender constructs the complete hierarchical `Parcel` and uploads it exactly once to save mobile data. The Relay Server duplicates the encrypted `Parcel` and executes the fan-out to all group members, operating with absolute zero-knowledge of the plaintext or the inner branch hierarchy.

8.2 Phase II: Voice Over Messages

Implementation of Voice-Over PQC compression algorithms to enable secure audio transmission.

9 Team Composition

Role	Name	University	Contribution
Team Leader	Protyasha Kundu	Indian Statistical Institute, Kolkata	Crypto Engine & Protocol Architect
Member 1	Biprarshi Biswas	Jadavpur University	Database & Storage Architect
Member 2	Sayandeepa Biswas	Indian Statistical Institute, Kolkata	Integration & Testing
Member 3	Shirsha Maitra	Heritage Institute of Technology	UI/UX & Backend Integration Architect

10 Mentors/Acknowledgments

Name	Role / Title	Organization	Expertise / Additional Info
Samrat Dasgupta	CEO	CryptIndo Labs	Industry Expert — Data & AI
Dr. Subhamoy Maitra	Professor & Head, Applied Statistics Unit	Indian Statistical Institute	-
Dr. Shashwat Raizada	Head of Business Development	CryptIndo Labs	Ex Commander (Sr. Systems Manager), WESEE, Indian Navy Cybersecurity Expert — Automotive & Defence

11 GitHub Repository

All the required codes of the Phase1 Prototype is in the Amortized_Quantum_Messaging repository

References

[1] M. Marlinspike and T. Perrin, "The X3DH Key Agreement Protocol," Signal, 2016. [Online]. Available: https://signal.org/docs/specifications/x3dh/.

[2] M. Marlinspike and T. Perrin, "The Double Ratchet Algorithm," Signal, 2016. [Online]. Available: https://signal.org/docs/specifications/doubleratchet/.

[3] Signal Technical Team, "The PQXDH Key Agreement Protocol," Signal, 2023. [Online]. Available: https://signal.org/docs/specifications/pqxdh/.

[4] E. Kret, "Quantum Resistance and the Signal Protocol," Signal Blog, Sep. 2023. [Online]. Available: https://signal.org/blog/pqxdh/.

[5] Apple Security Engineering and Architecture (SEAR), "iMessage with PQ3: The new state of the art in quantum-secure messaging at scale," Apple Security Research, Feb. 2024. [Online]. Available: https://security.apple.com/blog/imessage-pq3/.

[6] D. Stebila, "Security analysis of the iMessage PQ3 protocol," University of Waterloo / Apple, 2024. [Online]. Available: https://security.apple.com/assets/files/Security_analysis_of_the_iMessage_PQ3_protocol_Stebila.pdf.

[7] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, IETF, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446.

[8] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203)," NIST, Aug. 2024. [Online]. Available: https://csrc.nist.gov/pubs/fips/203/final.

[9] National Institute of Standards and Technology, "Module-Lattice-Based Digital Signature Standard (FIPS 204)," NIST, Aug. 2024. [Online]. Available: https://csrc.nist.gov/pubs/fips/204/final.